

Länkade listor

Vi har tidigare sett exempel på hur man kan deklarera en struct med pekare till ett objekt av samma typ. Troligen anar du att detta kan komma till användning i länkade listor:

Motiveringen till att använda länkade listor bör vara känd sedan tidigare kurser (dynamiskt storlek, snabbt att sätta in och ta bort element). Användning förutom som listor rätt och slätt, kan vara att låta en länkad lista utgöra en byggkloss till andra abstrakta datatyper som t.ex. hashtabeller och generella träd. Vilka operationer behövs på länkade listor? -Även detta bör vara bekant, men en hel del variation är möjlig. Här tas upp några rimliga exempel på operationer. I övrigt finns inget speciellt som gäller för länkade listor i C, det som tillkommer är implementationen, dvs hur C-koden för structer och funktioner ser ut. Naturligtvis kan även detta tillåtas variera inom vida gränser. Det som föreslås i detta paper är bara exempel.

Vi låter tills vidare det data som ska lagras i listan representeras av ett heltal (kallad 'key'), så kan vi sätta fokus på själva administrationen av listan.

Jag tänker visa en implementation med länkar åt båda hållen (ofta kallad dubbellänkad lista). Att ha den dubbellänkade gör att det blir några extra satser för att underhålla korrekta bakåt-länkar när man sätter in och tar bort länkar, men i gengäld kan en del andra operationer förenklas. Framförallt möjliggör det att skapa bakåtiteratörer (vilket dock inte implementeras här).

Ett förslag till liststruct:

```
typedef struct t_list {
    struct t_record *first, *last;
} t_list;
```

En post (dvs en länk):

```
typedef struct t_record {
    int key;
    struct t_record *next, *prev;
} t_record;
```

Några listoperationer:

- En funktion som skapar en lista. **In:** Inget **Ut:** En lista (eller NULL om minne saknas).
- En funktion som slänger bort en lista (detta är specifikt för C, minnet måste återlämnas). **In:** En giltig lista **Ut:** Inget
- En funktion som sätter in ett dataobjekt först. **In:** En giltig lista, data **Ut:** En lista med data insatt först i listan, och en indikering om det gick bra eller ej (1 eller 0).
- En funktion som sätter in ett dataobjekt på platsen före ett visst data **In:** En giltig lista, data att sätta in, data att placera före **Ut:** en lista med data insatt på angiven plats. Om det angivna dataobjektet inte finns placeras det nya först. Om det finns flera med samma 'key' är det odefinierat vilket som väljs, och en indikering om det gick bra eller ej (1 eller 0).
- En funktion som skapar en iterator för en lista. **In:** En giltig lista **Ut:** En iterator som representerar första elementet i listan, eller NULL om listan är tom.
- En funktion som ger nästa iterator. **In:** En giltig iterator **Ut:** En iterator som representerar nästa elementet i listan, eller NULL om inget mer element finns
- En funktion som tar bort ett visst dataobjekt. **In:** En giltig lista, en iterator i den listan **Ut:** En lista med angivet dataobjekt borttaget, nästa iterator (eller NULL om det var sista posten i listan)
- En funktion som ger det dataobjektet som en viss iterator representerar. **In:** En giltig iterator **Ut:** Ett dataobjekt

En iterator kan t.ex. implementeras så här:

```
typedef struct t_record t_iterator;
```

dvs, `t_iterator` är identisk med `t_record`. Det är visserligen en enkel, men ändå tillräcklig lösning. Fördelen är att man kan ha godtyckligt många iteratorer utan att skapa en avancerad konstruktion eller riska att slarva bort minne.

Nu gör vi först prototyperna för operationerna (interfacet):

- `t_list *create_list(void)`; Returnerar NULL om minne saknas annars listan
- `void destroy_list(t_list *l)`;
- `int insert_first(t_list *l, int key)`; Returnerar 0 om minne saknas annars 1
- `int insert_before(t_list *l, int key, int where)`; Returnerar 0 om minne saknas annars 1
- `t_iterator *first(t_list *l)`; Returnerar NULL om listan är tom
- `t_iterator *next(t_iterator *it)`; Returnerar NULL om it refererar till sista elementet i listan
- `t_iterator *remove_it(t_list *l, t_iterator *it)`; Returnerar en iterator till nästa element i listan (eller NULL om slut)
- `int get_data(t_iterator *it)`;

```
/* ----- Local functions: */
static void destroy_records(t_record *r)
{
    if (r) {
        destroy_records(r->next);
        free(r);
    }
}

static t_record *make_record(int key)
{
    t_record *r;
    r = (t_record*)malloc(sizeof(t_record));
    if (r) {
        r->key = key;
        r->next = r->prev = NULL;
    }
    return r;
}

static t_record *find_record(t_record *r, int key)
{
    if (r==NULL || r->key == key)
        return r;
    return find_record(r->next, key);
}

/* ----- Exported functions: */
t_list *create_list(void)
{
    return (t_list*)calloc(sizeof(t_list), 1);
}

void destroy_list(t_list *l)
{

```

```

    destroy_records(l->first);
    free(l);
}

int insert_first(t_list *l, int key)
{
    t_record *r;
    r = make_record(key);
    if (r) {
        if (l->first) {
            r->next = l->first;
            r->next->prev = r;
            l->first = r;
        } else
            l->first = l->last = r;
        return 1;
    } else
        return 0;
}

int insert_before(t_list *l, int key, int where)
{
    t_record *r, *nr;
    r = find_record(l->first, where);
    if (r) {
        if (r == l->first)
            return insert_first(l, key);
        nr = make_record(key);
        if (nr==NULL)
            return 0;
        nr->prev = r->prev;
        nr->next = r;
        nr->prev->next = nr;
        nr->next->prev = nr;
        return 1;
    } else
        return insert_first(l, key);
}

t_iterator *remove_it(t_list *l, t_iterator *it)
{
    t_iterator *next;
    next = it->next;
    if (it->prev)
        it->prev->next = it->next;
    else
        l->first = it->next;
    if (it->next)
        it->next->prev = it->prev;
    else
        l->last = it->prev;
    free(it);
    return next;
}

t_iterator *first(t_list *l)
{
    return l->first;
}

t_iterator *next(t_iterator *it)
{
    return it->next;
}

```

```
int get_data(t_iterator *it)
{
    return it->key;
}
```

Som sagt man kan tänka sig fler matnyttiga funktioner, t.ex. "*sätt in ett dataobjekt sist*" eller "*ge dataobjektet på plats i*", dvs indexering eller varför inte skapa en *bakåt-iterator*. Prova gärna att först skriva prototyper sedan implementationer för dessa.

Fler ideer: gör om detta till en *mängd*. Vilka implikationer får det på ADT-beskrivningen? Implementationen?

Vilket ordo har de olika operationerna?

Finns det någon av funktionerna som du tycker skulle bli bättre om man gör om interfacet? Implementationen?